

Terraform - Infrastructure as Code

Utilité dans le projet

Terraform permet de **gérer l'infrastructure Kubernetes de manière déclarative** :

-  Infrastructure versionnée dans Git
-  Reproductibilité garantie (dev = prod)
-  Visualisation des changements avant application
-  State management (suivi de l'état)
-  Rollback facile vers versions précédentes

Comparaison : kubectl vs Terraform

Aspect	kubectl apply	Terraform
Approche	Impérative	Déclarative
State	✗ Non géré	✓ terraform.tfstate
Preview	✗ Non	✓ terraform plan
Modules	✗ Non	✓ Réutilisables
Multi-env	Manuel	✓ dev/prod séparés
Rollback	Manuel	✓ Automatique

Architecture Terraform

```
terraform/
├─ environments/
│   ├── dev/                # Environnement développement
│   │   ├── main.tf         # Appelle les modules
│   │   ├── variables.tf    # Variables d'entrée
│   │   ├── outputs.tf      # Valeurs de sortie
│   │   ├── locals.tf       # Calculs locaux
│   │   └─ terraform.tfvars # Valeurs spécifiques
│   └─ prod/                # Environnement production
│       └─ ... (même structure)
└─ modules/                # Modules réutilisables
    ├── namespace/         # Crée le namespace K8s
    ├── mongodb/           # Déploie MongoDB
    ├── backend/           # Déploie l'API
    └─ frontend/           # Déploie React
```

Modules expliqués

1. Module Namespace

Fichier : `terraform/modules/namespace/main.tf`

```
resource "kubernetes_namespace_v1" "main" {
  metadata {
    name = var.namespace_name
    labels = {
      "app.kubernetes.io/managed-by" = "terraform"
      "environment" = var.environment
    }
  }
}

# Optionnel : Quotas de ressources
resource "kubernetes_resource_quota_v1" "namespace_quota" {
  count = var.enable_resource_quota ? 1 : 0

  metadata {
    name      = "${var.namespace_name}-quota"
    namespace = kubernetes_namespace_v1.main.metadata[0].name
  }

  spec {
    hard = {
      "requests.cpu"      = "4"
      "requests.memory"   = "4Gi"
      "pods"              = "20"
    }
  }
}
```

Ce qui est créé :

- Namespace Kubernetes
- Labels pour organisation
- Quotas optionnels (prod)

2. Module MongoDB

Fichier : `terraform/modules/mongodb/main.tf`

```
# Secret pour credentials
resource "kubernetes_secret_v1" "mongo" {
  metadata {
    name      = "mongo-secret"
    namespace = var.namespace
  }

  data = {
    mongodb-password = var.mongo_password
  }
}

# PVC pour stockage persistant
resource "kubernetes_persistent_volume_claim_v1" "mongo" {
  metadata {
    name      = "mongo-pvc"
    namespace = var.namespace
  }

  spec {
    access_modes = ["ReadWriteOnce"]
    resources {
      requests = {
        storage = var.storage_size # 1Gi (dev), 5Gi (prod)
      }
    }
  }
}

# Deployment MongoDB
resource "kubernetes_deployment_v1" "mongo" {
  metadata {
    name      = "mongo-deployment"
    namespace = var.namespace
  }

  spec {
    replicas = var.replicas

    template {
      spec {
        container {
          name = "mongodb"
          image = "mongo:7"

          port {
            container_port = 27017
          }

          # Volume mount pour persistence
          volume_mount {
            name = "mongo-data"
          }
        }
      }
    }
  }
}
```

```

        mount_path = "/data/db"
    }

    # Health checks
    liveness_probe {
        exec {
            command = ["mongosh", "--eval", "db.adminCommand('ping')"]
        }
        initial_delay_seconds = 30
    }
}

volume {
    name = "mongo-data"
    persistent_volume_claim {
        claim_name = kubernetes_persistent_volume_claim_v1.mongo.metadata[0].name
    }
}
}
}
}

# Service ClusterIP
resource "kubernetes_service_v1" "mongo" {
    metadata {
        name      = "mongo-service"
        namespace = var.namespace
    }

    spec {
        type = "ClusterIP"
        selector = {
            app = "mongo"
        }
        port {
            port          = 27017
            target_port = 27017
        }
    }
}
}

```

Ce qui est créé :

- Secret avec password MongoDB
- PVC 1Gi (dev) ou 5Gi (prod)
- Deployment avec health checks
- Service ClusterIP

3. Module Backend

Similaire à MongoDB avec :

- ConfigMap (NODE_ENV, PORT)
- Secret (JWT_SECRET, MONGO_URI)
- Deployment avec health checks HTTP
- Service ClusterIP port 5000

4. Module Frontend**Similaire aux autres avec :**

- ConfigMap (VITE_API_URL)
- Deployment Nginx
- Service NodePort 30080

 Environnements**Dev (`environments/dev/`)**

```
# terraform.tfvars
namespace_name = "devops-portfolio-dev"
environment    = "dev"

# Resources limitées
backend_replicas = 1
mongo_storage_size = "1Gi"
backend_cpu_request = "50m"
backend_memory_request = "64Mi"

# Image pull toujours latest
image_pull_policy = "Always"

# Monitoring désactivé
enable_monitoring = false
```

Prod (`environments/prod/`)

```
# terraform.tfvars
namespace_name = "devops-portfolio"
environment     = "prod"

# High Availability
backend_replicas = 2 # Minimum 2
frontend_replicas = 2

# Resources augmentées
mongo_storage_size = "5Gi"
backend_cpu_request = "200m"
backend_memory_request = "256Mi"

# Image pull avec cache
image_pull_policy = "IfNotPresent"

# Monitoring activé
enable_monitoring = true
enable_resource_quota = true
```

Utilisation

1. Configuration initiale

```
cd terraform/environments/dev

# Copier l'exemple
cp terraform.tfvars.example terraform.tfvars

# Éditer les secrets
nano terraform.tfvars
```

terraform.tfvars minimum :

```
jwt_secret      = "votre-secret-32-caracteres-minimum"
mongo_password = "votre-password-securise"
```

2. Initialisation

```
terraform init
```

Ce qui se passe :

- Télécharge le provider Kubernetes
- Initialise les modules

- Prépare le state local

3. Planification

```
terraform plan -out=tfplan
```

Output exemple :

```
Terraform will perform the following actions:

# module.namespace.kubernetes_namespace_v1.main will be created
+ resource "kubernetes_namespace_v1" "main" {
  + metadata {
    + name = "devops-portfolio-dev"
  }
}

# module.mongodb.kubernetes_deployment_v1.mongo will be created
...

Plan: 15 to add, 0 to change, 0 to destroy.
```

4. Application

```
terraform apply tfplan
```

Ce qui est créé :

- 1 Namespace
- 3 Secrets
- 2 ConfigMaps
- 1 PVC
- 3 Deployments
- 3 Services

5. Vérification

```
kubectl get all -n devops-portfolio-dev

# Ou voir les outputs Terraform
terraform output
```


State Management

terraform.tfstate

Fichier JSON qui contient :

- État actuel de l'infrastructure
- Mapping ressources Terraform ↔ Kubernetes
- Métadonnées et dépendances

 **Important** : Ne JAMAIS modifier manuellement

Commandes state

```
# Lister les ressources
terraform state list

# Voir une ressource
terraform state show module.mongodb.kubernetes_deployment_v1.mongo

# Supprimer du state (sans détruire)
terraform state rm module.frontend

# Pull state distant
terraform state pull > backup.tfstate
```

Workflows courants

Mettre à jour les réplicas

```
# Dans terraform.tfvars
backend_replicas = 3

# Appliquer
terraform plan
terraform apply
```

Changer l'image Docker

```
# Dans terraform.tfvars
backend_image = "lims4/backend:v2.0.0"

terraform apply
```

Passer de dev à prod

```
cd ../prod

# Configurer prod
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars

# Déployer
terraform init
terraform plan
terraform apply
```

Destruction

Supprimer tout

```
terraform destroy
```

⚠ **Attention** : Supprime TOUTES les ressources (namespace, PVC, pods...)

Supprimer une ressource spécifique

```
terraform destroy -target=module.frontend
```

Avantages Terraform

1. Reproductibilité

```
# Même résultat partout
terraform apply # Dev
terraform apply # Prod
```

2. Preview des changements

```
# Avant d'appliquer, voir ce qui va changer
terraform plan

# Output:
# ~ update in-place
# + create
# - destroy
```

3. Rollback facile

```
# Revenir à l'état précédent
terraform state pull > backup.tfstate
terraform apply backup.tfstate
```

4. Documentation vivante

Le code Terraform EST la documentation de l'infrastructure

5. Validation

```
# Valider la syntaxe
terraform validate

# Formater le code
terraform fmt -recursive
```

Sécurité

Secrets

- ✓ Variables `sensitive = true`
- ✓ terraform.tfvars dans .gitignore
- ✓ Validation longueur minimale

Best practices

```
variable "jwt_secret" {  
  type      = string  
  sensitive = true  
  
  validation {  
    condition     = length(var.jwt_secret) >= 32  
    error_message = "JWT secret doit faire 32+ caractères"  
  }  
}
```



Intégration CI/CD

Jenkinsfile (futur)

```
stage('Terraform Plan') {  
  steps {  
    sh '''  
      cd terraform/environments/prod  
      terraform init  
      terraform plan -out=tfplan  
    '''  
  }  
}  
  
stage('Terraform Apply') {  
  when {  
    branch 'main'  
  }  
  steps {  
    sh '''  
      cd terraform/environments/prod  
      terraform apply -auto-approve tfplan  
    '''  
  }  
}
```



Concepts clés

- **Provider** : Interface avec Kubernetes
- **Resource** : Objet K8s (Deployment, Service...)
- **Module** : Groupe de ressources réutilisables
- **Variable** : Paramètre d'entrée
- **Output** : Valeur de sortie
- **Local** : Variable calculée
- **State** : État actuel de l'infra

- **Data source** : Lecture d'infos externes

Documentation complète : [terraform/README.md](#)